Java DSA: 10 Array Practice Questions

Simple English study notes adapted from the attached Hindi YouTube lecture transcript (~76 minutes). The instructor demonstrates questions 1-7 and assigns questions 8-10 for practice.

Learning pattern: understand the problem, explain the approach, implement reusable Java methods, test with input, and analyze time and extra space complexity.

1. Find the Largest Element

Start with the first element as largest. Scan the remaining elements and replace largest whenever a bigger value is found. Example: [20,45,23,11,67] -> 67.

```
static int largest(int[] a) {
    if (a.length == 0) throw new IllegalArgumentException("Empty array");
    int max = a[0];
    for (int i = 1; i < a.length; i++)
        if (a[i] > max) max = a[i];
    return max;
}
```

Time: O(n). Extra space: O(1). Initializing from a[0] also works for arrays containing only negative values.

2. Reverse an Array

Two approaches in the lecture: print elements from the last index to the first, or swap opposite elements until the middle. Swapping modifies the original array; printing backward does not.

```
static void reverse(int[] a) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;
        left++; right--;
    }
}
```

Example: [90,80,10,20,50,75] -> [75,50,20,10,80,90]. Time: O(n). Extra space: O(1). A separate reversed array uses O(n) extra space.

3. Sum of All Array Elements

Use an accumulator initialized to zero. Add each array element to it. Example: [1,2,3,4,5] -> 15.

```
static long sum(int[] a) {
    long total = 0;
    for (int value : a) total += value;
    return total;
}
```

Time: O(n). Extra space: O(1). The lecture also mentions n*(n+1)/2, which applies specifically to the integers from 1 through n, not arbitrary arrays.

4. Count Occurrences of a Target

Read the target, scan the array, and increment a counter whenever an element equals the target. Example: [12,6,6,6,9], target 6 -> 3.

```
static int countOccurrences(int[] a, int target) {
    int count = 0;
    for (int value : a)
        if (value == target) count++;
    return count;
```

```
}
```

Time: O(n). Extra space: O(1). If the target does not appear, return zero.

5. Find the Second Largest Element

The lecture maintains two values: the largest and the second-largest. Here second-largest means the second distinct largest value; duplicate copies of the maximum do not count.

```
static int secondLargestDistinct(int[] a) {
    Integer first = null, second = null;
    for (int x : a) {
        if (first == null || x > first) {
            second = first; first = x;
        } else if (x < first && (second == null || x > second)) {
            second = x;
        }
    }
    if (second == null) throw new IllegalArgumentException(
        "Need two distinct values");
    return second;
}
```

Example: [90,80,10,20,50,75] -> 80. Time: O(n). Extra space: O(1). Handle arrays with fewer than two distinct values.

6. Check Whether an Array Is Sorted

Compare every element with the previous one. If any later element is smaller, the array is not sorted in nondecreasing (ascending) order.

```
static boolean isSortedAscending(int[] a) {
    for (int i = 1; i < a.length; i++)
        if (a[i] < a[i - 1]) return false;
    return true;
}
```

Example: [1,2,2,7] -> true; [1,5,3] -> false. Time: O(n) worst case, O(1) extra space.

7. Move All Zeros to the End

The lecture moves zero values to the right while keeping nonzero values together. A write pointer provides an in-place, stable solution: first copy nonzero elements forward, then fill the rest with zeros.

```
static void moveZerosToEnd(int[] a) {
    int write = 0;
    for (int value : a)
        if (value != 0) a[write++] = value;
    while (write < a.length) a[write++] = 0;
}
```

Example: [0,1,0,3,12] -> [1,3,12,0,0]. Time: O(n). Extra space: O(1).

Questions 8-10: Homework from the Lecture

8. Remove duplicates from a sorted array. Suggested direction: two pointers; write each new distinct value once. Decide whether to return the new logical length.

9. Find the missing number. The lecture describes numbers from 1 to n with one missing. Suggested direction: expected sum minus actual sum; carefully define input length and range.

10. Rotate an array right by k positions. Example: [1,2,3,4,5], k=2 -> [4,5,1,2,3]. Suggested direction: normalize k using k % n and use reversal steps or a temporary array.

Reusable Java Input Template

```
import java.util.Scanner;

public class ArrayPractice {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            System.out.print("Array size: ");
            int n = sc.nextInt();
            if (n < 0) throw new IllegalArgumentException("Negative size");
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
            if (n > 0) System.out.println(largest(arr));
        }
    }
    static int largest(int[] a) {
        int max = a[0];
        for (int v : a) if (v > max) max = v;
        return max;
    }
}
```

Java Clarifications and Interview Notes

Java is pass-by-value. When an array is passed to a method, the copied value is a reference to the same array object; modifying its elements is visible to the caller. Reassigning the local parameter to a new array does not change the caller variable. The lecture demonstrates this distinction during the reverse-array example.

Complexity: a single full traversal is O(n), even if it contains constant-time comparisons. In-place reversing and moving zeros use O(1) auxiliary space. A new output array uses O(n) space.

Test edge cases: empty array, one element, all equal values, all negative values, zeros only, already sorted, reverse sorted, repeated maximum, and large integer sums.

Quick Revision

Largest -> keep running maximum.

Reverse -> swap left and right until they meet.

Sum -> running accumulator.

Count -> increment only on target match.

Second largest -> maintain two distinct maxima.

Sorted -> check adjacent order.

Move zeros -> write nonzero values, then fill zeros.

Homework -> sorted deduplication, missing number, rotate right by k.

Source: User-provided Hindi YouTube transcript. These are structured study notes, not a verbatim transcript. Code is organized for readability; edge-case checks and technical clarifications are included where useful.